



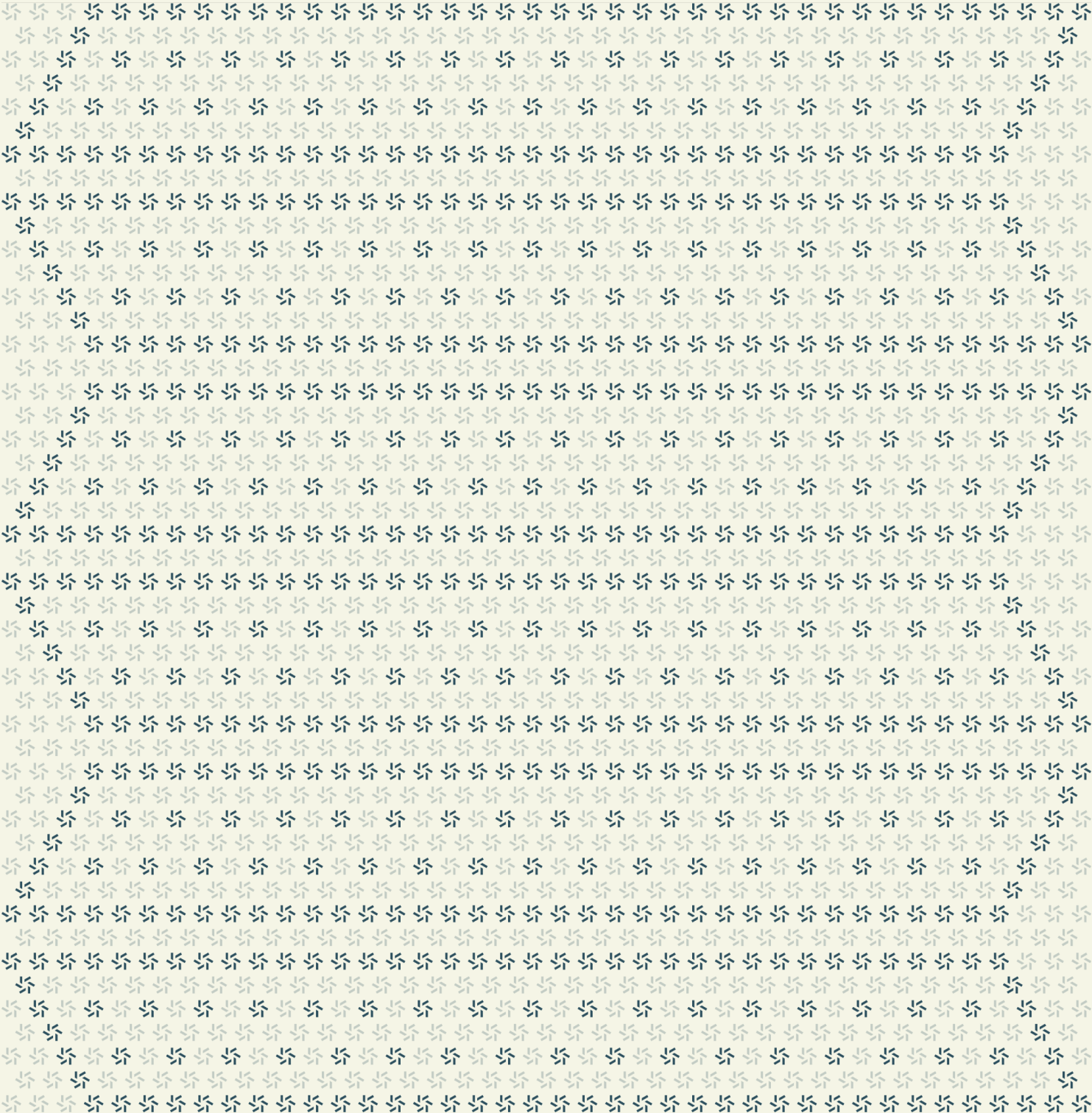
Prepared for
Junbug
Bulk Trade

Prepared by
Ziling Chen
Xuehao Guo
Oliver Murray
Zellic

July 23, 2026

Bulk Program

Solana Application Security Assessment



Contents

About Zellic	4
1. Overview	5
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	7
2.1. About Bulk Program	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	11
3.1. The admin key can bypass FROST authorization and drain every vault	11
3.2. Reversible pre-deposit shutdown enables double claims against post-migration liquidity	15
3.3. Migrated pre-deposit accounts cannot reclaim rent	19
3.4. Nested tuple layout is not stable across program upgrades	22
3.5. Initialization does not verify control of configured authority keys	24
3.6. <code>max_balance</code> does not impose a protocol-wide deposit cap	26

4.	Discussion	28
4.1.	Code quality and implementation simplicity	28
4.2.	Narrow mutable borrows and reference scopes	28
4.3.	Remove the unused <code>set_frost_group_key</code> helper	28
5.	Threat Model	29
5.1.	Program: <code>bulk-program</code>	29
6.	Assessment Results	57
6.1.	Disclaimer	57

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) [worldwide](#) in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io [and](#) follow [@zellic_io](#) [on Twitter](#). If you are interested in partnering with Zellic, contact us at hello@zellic.io [.](#)



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Bulk Trade from July 13th to July 16th, 2026. During this engagement, Zellic reviewed Bulk Program's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Can an unprivileged party withdraw SPL tokens or move them to an unintended vault or token account?
 - Are PDA, signer, mint, and token-account validations correct?
 - Can admin or FROST authority management, including initialization and key rotation, be abused to gain unauthorized custody access?
 - Can migration from the pre-deposit phase to open deposits safely preserve user balances, prevent double claims, and avoid locking funds?
 - Are low-level Rust operations, including unsafe state deserialization and account borrows, memory-safe and resilient to account aliasing or CPI-related bugs?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Bulk Program programs, we discovered six findings. No critical issues were found. One finding was of high impact, one was of medium impact, one was of low impact, and the remaining findings were informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Bulk Trade in the Discussion section ([4. 7](#)).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	1
Medium	1
Low	1
Informational	3



2. Introduction

2.1. About Bulk Program

Bulk Trade contributed the following description of Bulk Program:

The program provides SPL-token custody vaults for the Bulk validator network. Anyone may deposit, while withdrawals require a transaction signed by the network's FROST group key. Threshold signing occurs off-chain, and the on-chain program enforces custody and account-validation checks.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the programs.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood.

We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped programs itself. These observations — found in the Discussion ([4.7](#)) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

Bulk Program Programs

Type	Solana
Platform	Solana
Target	bulk-program
Repository	https://github.com/Bulk-trade/bulk-program ↗
Version	4b343b465eca6ed2dc32675483badf7a9296cf2c
Programs	programs/src/**/*.*rs

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 1.2 person-weeks. The assessment was conducted by three consultants over the course of four calendar days.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
🔗 Engagement Manager
jacob@zellic.io 🔗

The following consultants were engaged to conduct the assessment:

Ziling Chen
🔗 Engineer
ziling@zellic.io 🔗

Xuehao Guo
🔗 Engineer
guo@zellic.io 🔗

Oliver Murray
🔗 Engineer
oliver@zellic.io 🔗

2.5. Project Timeline

The key dates of the engagement are detailed below.

July 13, 2025 Kick-off call

July 16, 2025 Start of primary review period

July 16, 2025 End of primary review period

3. Detailed Findings

3.1. The admin key can bypass FROST authorization and drain every vault

Target	Bulk		
Category	Protocol Risks	Severity	Critical
Likelihood	Low	Impact	High

Description

README-REVIEW.md presents FROST threshold signing as the withdrawal security boundary. It states that the corresponding private key is never assembled and that a valid signature can only be produced when at least $2f + 1$ validators participate. However, the on-chain authorization model allows the admin_key to replace frost_group_key unilaterally with any address.

The authorization helper permits either the existing FROST group or the administrator to perform a group-key rotation:

```
pub fn check_frost_or_admin_signer(state: &Bulk, signer: &AccountView) ->
    ProgramResult {
    let signer = signer.address();
    if state.frost_group_key != *signer && state.admin_key != *signer {
        return Err(ProgramError::Custom(1));
    }
    Ok(())
}
```

The process_update_frost_group instruction accepts an arbitrary 32-byte address and installs it immediately. It does not require authorization from the existing validator threshold, proof that the incoming group controls the new key, a time delay, or a separate acceptance step:

```
pub fn process_update_frost_group(
    program_id: &Address,
    accounts: &mut [AccountView],
    data: &[u8],
) -> ProgramResult {
    let [signer, state] = accounts else {
        return Err(ProgramError::NotEnoughAccountKeys);
    };

    is_signer(signer)?;
    check_bulk_state(state, program_id)?;
```

```
let state = unsafe { load_mut::<Bulk>(state.borrow_unchecked_mut())? };
// Replacing with 'check_frost' after since in production currently frost
// is also a admin key, so when the main set is activated we use the admin
// key as a frost to move into prod frost flow.
check_frost_or_admin_signer(state, signer)?;

if data.len() != 32 {
    return Err(ProgramError::InvalidInstructionData);
}

let frost_group_key = Address::from(
    <[u8;
32]>::try_from(data).map_err(|_| ProgramError::InvalidInstructionData)?,
);

state.frost_group_key = frost_group_key;

Ok(())
}
```

The test suite identifies this authority as intentional break-glass recovery:

```
#[test]
fn update_frost_group_allows_admin_signer() {
    let mut harness = Harness::new_initialized();
    let new_frost = Address::new_unique();

    // Admin is an authorized signer for frost-group rotation (break-glass
    recovery).
    harness.update_frost_group(harness.admin, new_frost, &[Check::success()]);

    // ...
}
```

A break-glass objective does not remove the custody risk. The mechanism is available while the validator network is healthy, has no recovery-specific restrictions, and gives one administrator the effective withdrawal authority that README-REVIEW.md attributes exclusively to a threshold of validators. A malicious or compromised administrator can set `frost_group_key` to an ordinary key they control.

The attacker-controlled key then passes the only authorization check in `process_withdraw`. The destination token account is constrained by mint but not by an on-chain withdrawal entitlement, because the program relies on the FROST network to authorize recipients and amounts:

```
check_bulk_state(state, program_id)?;
```

```
{
    let bulk_state = unsafe { load::<Bulk>(state.borrow_unchecked())? };
    check_frost_signer(bulk_state, payer)?;
}

// ...

let user_ta =
    pinocchio_token::state::Account::from_account_view(user_token_account)?;
if user_ta.mint() != mint.address() {
    return Err(ProgramError::InvalidAccountData);
}

// ...

Transfer::new(vault_token_account, user_token_account, vault,
    amount).invoke_signed(&[
    pinocchio::cpi::Signer::from(&[
        pinocchio::cpi::Seed::from(Vault::SEED.as_ref()),
        pinocchio::cpi::Seed::from(mint.address().as_ref()),
        pinocchio::cpi::Seed::from(&[bump]),
    ]),
    ])?;
```

Regular deposits and predeposits are transferred into the same vault token accounts:

```
// program/src/instructions/deposit.rs
Transfer::new(user_token_account, vault_token_account, payer,
    amount).invoke()?;

// program/src/instructions/pre_deposit/deposit.rs
Transfer::new(user_token_account, vault_token_account, payer, amount).invoke()
```

Consequently, the administrator can rotate the group key and execute the first malicious withdrawal in the same atomic Solana transaction. They can then repeat the withdrawal for every configured vault, draining both regular deposits and predeposits without participation from the validator threshold.

Impact

A malicious administrator or an attacker who compromises the administrator key can replace the FROST group key and withdraw all token balances held across the protocol vaults. This bypasses the validator threshold and places both regular deposits and predeposits at risk.

Recommendations

Require the existing FROST group to authorize every group-key rotation and require the incoming group to prove control of the new key before activation.

Remediation

This issue has been acknowledged by Bulk Trade, and a fix was implemented in commit [f958edc2](#).

3.2. Reversible pre-deposit shutdown enables double claims against post-migration liquidity

Target	PreDepositConfig		
Category	Business Logic	Severity	High
Likelihood	Low	Impact	Medium

Description

The pre-deposit migration flow relies on disabling vault activity before the off-chain Bulk network credits the corresponding user balances. However, the shutdown does not create a terminal migration state. The `process_disable_all_vault_activity` instruction only calls `disable_all_vault_activity` on the shared configuration:

```
pub fn process_disable_all_vault_activity(
    program_id: &Address,
    accounts: &mut [AccountView],
    data: &[u8],
) -> ProgramResult {
    // ...

    let config =
        unsafe {
            load_mut:::<PreDepositConfig>(pre_deposit_config.borrow_unchecked_mut())?
        };

    config.disable_all_vault_activity();

    Ok(())
}
```

The configuration method only clears the active flag for each configured vault. It does not record that migration has started or completed:

```
pub fn disable_all_vault_activity(&mut self) {
    let n = (self.vault_count as usize).min(self.vault_flags.len());
    for flags in &mut self.vault_flags[..n] {
        *flags &= !VAULT_FLAG_ACTIVE;
    }
}
```

The active flag remains a valid configuration value after the shutdown:

```
pub const VAULT_FLAG_ACTIVE: u8 = 1 << 0;
pub const VAULT_FLAGS_VALID: u8 = VAULT_FLAG_ACTIVE;

pub fn parse_vault_flags(flags: u8) -> Result<u8, ProgramError> {
    if flags & !VAULT_FLAGS_VALID != 0 {
        return Err(ProgramError::InvalidInstructionData);
    }
    Ok(flags)
}
```

An admin or the FROST group can therefore call `process_set_vault_flags` after migration and restore `VAULT_FLAG_ACTIVE`:

```
let flag = PreDepositConfig::parse_vault_flags(data[0])?;

// ...

{
    let bulk_state = unsafe { load::<Bulk>(state.borrow_unchecked())? };
    check_frost_or_admin_signer(bulk_state, signer)?;
}

// ...

let idx = config
    .vault_index(vault.address())
    .ok_or(ProgramError::InvalidAccountData)?;

config.set_flags(idx, flag);
```

The migration withdrawal path does not receive or update a `PreDeposit` account. It transfers tokens from the vault based only on FROST authorization:

```
let [
    payer,
    user_token_account,
    vault,
    vault_token_account,
    mint,
    state,
    token_program,
    ..,
] = accounts
```



```

else {
    return Err(ProgramError::NotEnoughAccountKeys);
};

// ...

Transfer::new(vault_token_account, user_token_account, vault,
amount).invoke_signed(&[
    pinocchio::cpi::Signer::from(&[
        pinocchio::cpi::Seed::from(Vault::SEED.as_ref()),
        pinocchio::cpi::Seed::from(mint.address().as_ref()),
        pinocchio::cpi::Seed::from(&[bump]),
    ]),
])?;

```

Consequently, each user's original `PreDepositEntry` remains intact after the off-chain system credits the migrated balance. Once a vault is reactivated, `process_pre_withdraw` accepts the stale entry and transfers tokens from the vault:

```

if !config.has_flag(vault_idx, VAULT_FLAG_ACTIVE) {
    return Err(ProgramError::Custom(245));
}

// ...

let state = unsafe {
    load_mut::<PreDeposit>(pre_deposit.borrow_unchecked_mut())? };

// ...

withdraw_from_entry(state, vault.address(), amount)?;

Transfer::new(vault_token_account, user_token_account, vault,
amount).invoke_signed(&[
    pinocchio::cpi::Signer::from(&[
        pinocchio::cpi::Seed::from(Vault::SEED.as_ref()),
        pinocchio::cpi::Seed::from(mint.address().as_ref()),
        pinocchio::cpi::Seed::from(&[bump]),
    ]),
])

```

Regular deposits remain available independently of the pre-deposit configuration and transfer tokens into the same vault token account:

```

let [

```

```
payer,  
user_token_account,  
vault,  
mint,  
vault_token_account,  
token_program,  
] = accounts  
else {  
    return Err(ProgramError::NotEnoughAccountKeys);  
};  
  
// ...  
  
Transfer::new(user_token_account, vault_token_account, payer,  
amount).invoke()?;
```

A mistaken or malicious reactivation therefore makes both the migrated off-chain balance and the stale on-chain withdrawal entitlement usable. If migration already removed the original tokens, later regular deposits replenish the shared vault token account and can be withdrawn by the holders of stale pre-deposit entries.

Impact

After a vault is reactivated, predepositors can claim balances that were already credited during migration. These duplicate claims can consume post-migration liquidity belonging to regular depositors, create a vault shortfall, and cause depositor losses.

Recommendations

Add an irreversible migrated state to `PreDepositConfig` and permanently reject vault reactivation and legacy predeposit withdrawals after migration is finalized.

Remediation

This issue has been acknowledged by Bulk Trade, and a fix was implemented in commit [6cd7391c](#).

3.3. Migrated pre-deposit accounts cannot reclaim rent

Target	PreDeposit		
Category	Completeness	Severity	Low
Likelihood	High	Impact	Low

Description

The intended migration flow disables all pre-deposit vault activity before balances are migrated to the off-chain Bulk network. However, users can close their PreDeposit accounts only when entry_count is zero:

```
let state = unsafe { load::<PreDeposit>(pre_deposit.borrow_unchecked())? };
if state.user != *signer.address() {
    return Err(ProgramError::InvalidAccountOwner);
}
if state.entry_count != 0 {
    return Err(ProgramError::InsufficientFunds);
}

let signer_lamports = signer
    .lamports()
    .checked_add(pre_deposit.lamports())
    .ok_or(ProgramError::ArithmeticOverflow)?;
signer.set_lamports(signer_lamports);
pre_deposit.set_lamports(0);
pre_deposit.close();
```

The only code path that decrements entry_count is withdraw_from_entry, which does so after a user fully withdraws one vault entry:

```
let remaining = current - amount;
if remaining == 0 {
    let last = entry_count
        .checked_sub(1)
        .ok_or(ProgramError::InvalidAccountData)?;
    slots[idx] = slots[last];
    slots[last] = PreDepositEntry::default();
    state.entry_count = state
        .entry_count
        .checked_sub(1)
```

```
        .ok_or(ProgramError::InvalidAccountData)?;
    } else {
        slots[idx].amount = remaining.to_le_bytes();
    }
}
```

The pre-deposit withdrawal path rejects inactive vaults before it loads or modifies the user's PreDeposit account:

```
let config = unsafe {
    load::<PreDepositConfig>(pre_deposit_config.borrow_unchecked())? };
let vault_idx = config
    .vault_index(vault.address())
    .ok_or(ProgramError::InvalidAccountData)?;

if !config.has_flag(vault_idx, VAULT_FLAG_ACTIVE) {
    return Err(ProgramError::Custom(245));
}
```

The migration shutdown clears every active flag but does not clear user entries or introduce an alternative settlement state:

```
pub fn disable_all_vault_activity(&mut self) {
    let n = (self.vault_count as usize).min(self.vault_flags.len());
    for flags in &mut self.vault_flags[..n] {
        *flags &= !VAULT_FLAG_ACTIVE;
    }
}
```

As a result, every user with a nonempty PreDeposit account at the migration cutoff retains a nonzero entry_count, cannot use process_pre_withdraw to reduce it, and cannot use process_close_account to reclaim the account's rent-exempt lamports. The only available workaround is to reactivate the affected vaults and return the underlying tokens so that users can withdraw them, which contradicts the intended migration and reintroduces stale withdrawal claims.

Impact

Each user who retains a pre-deposit entry when migration begins is left with an allocated 681-byte account whose rent-exempt lamports cannot be reclaimed. The obsolete account state persists after migration, although deposited token principal is not directly affected.

Recommendations

Add a user-authorized `close_migrated_account` instruction that clears obsolete entries and closes the PDA only after the program reaches an irreversible migrated state.

Remediation

This issue has been acknowledged by Bulk Trade, and a fix was implemented in commit [673f835b](#).

3.4. Nested tuple layout is not stable across program upgrades

Target	PreDepositConfig		
Category	Soundness	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The program reads and writes `PreDepositConfig` directly from persistent account data through zero-copy pointer casts. Although `PreDepositConfig` uses `#[repr(C)]`, this representation only determines the layout of its direct fields. It does not recursively apply the C representation to nested types.

The `vault_min_max_deposit` field is an array of tuples, and tuples retain the default Rust representation:

```
#[repr(C)]
pub struct PreDepositConfig {
    pub discriminator: [u8; 8],
    pub vaults: [Address; 16],
    pub vault_flags: [u8; 16],
    /// Per vault: `(min_per_tx, max_balance)` as little-endian u64 bytes.
    `0` = unset.
    pub vault_min_max_deposit: [([u8; 8], [u8; 8]); 16],
    pub vault_count: u8,
}
```

Rust does not define a stable ABI for the offsets or ordering of fields within a tuple. The array elements remain ordered and contiguous, but the layout of the two `[u8; 8]` fields inside each tuple is not guaranteed across compiler versions or build configurations.

The accessors rely on tuple index zero representing `min_per_tx` and tuple index one representing `max_balance`:

```
pub fn min_per_tx(&self, idx: usize) -> u64 {
    u64::from_le_bytes(self.vault_min_max_deposit[idx].0)
}

pub fn max_balance(&self, idx: usize) -> u64 {
    u64::from_le_bytes(self.vault_min_max_deposit[idx].1)
}
```

```
pub fn set_min_max_deposit(
    &mut self,
    idx: usize,
    min_per_tx: u64,
    max_balance: u64,
) -> Result<(), ProgramError> {
    if min_per_tx != 0 && max_balance != 0 && min_per_tx > max_balance {
        return Err(ProgramError::InvalidInstructionData);
    }
    self.vault_min_max_deposit[idx] =
        (min_per_tx.to_le_bytes(), max_balance.to_le_bytes());
    Ok(())
}
```

A single program build reads and writes this representation consistently, so the audited build does not immediately misinterpret the values. However, the account bytes survive program upgrades. If a later compiler assigns different tuple-field offsets, upgraded code could interpret previously stored minimum values as maximum values, or vice versa. Because both tuple fields have the same size and alignment, the existing account-length and alignment checks would not detect such a change.

Impact

A program upgrade built with a different tuple layout could misinterpret persisted predeposit limits. This could cause valid deposits to be rejected or apply unintended limits while the predeposit configuration remains in use.

Recommendations

Replace the nested tuple with a named `#[repr(C)]` type that preserves the existing 16-byte wire format, and add a byte-level layout test that asserts the exact field offsets and total size.

Remediation

This issue has been acknowledged by Bulk Trade, and a fix was implemented in commit [712ce884](#).

3.5. Initialization does not verify control of configured authority keys

Target	Bulk		
Category	Code Maturity	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The `process_initialize` instruction accepts the FROST group key and administrator key as raw instruction data. The instruction requires the payer to sign and verifies that the payer is the program upgrade authority, but neither configured authority is required to participate in initialization or otherwise prove control of its corresponding private key.

The initializer decodes the two authority addresses, authenticates only the upgrade authority, and then stores the supplied values:

```
is_signer(payer)?;

// ...

let frost_group_key = Address::from(
    <[u8; 32]>::try_from(&data[..32])
    .map_err(|_| ProgramError::InvalidInstructionData)?,
);

let admin_key = Address::from(
    <[u8; 32]>::try_from(&data[32..64])
    .map_err(|_| ProgramError::InvalidInstructionData)?,
);

// ...

if upgrade_authority != *payer.address() {
    return Err(ProgramError::IllegalOwner);
}

// ...

bulk_state.frost_group_key = frost_group_key;
bulk_state.admin_key = admin_key;
```

This behavior does not allow an arbitrary caller to initialize the program because the payer must

be the authenticated upgrade authority. The remaining risk is operational. A typographical error, stale deployment configuration, or use of an address whose key material is unavailable could assign privileged access to an unintended address or leave the intended operators unable to exercise their privileges.

The FROST group key authorizes withdrawals, while either configured key can authorize FROST rotation and vault initialization. If only the FROST key is incorrect, the valid administrator key can replace it; if only the administrator key is incorrect, the valid FROST key can continue operating. If neither key is controlled, `process_initialize_vault` also rejects setup, so a normal deployment becomes unusable before vaults can accept user funds. This makes the finding a configuration-integrity and deployment-availability concern rather than a direct funds-at-risk condition.

Impact

Incorrect authority addresses can delay or prevent protocol activation, remove the intended administrative recovery role, or assign an operational role to an unintended address. If neither configured key is controlled, vault initialization cannot be completed.

Recommendations

Use a staged initialization flow in which the administrator and FROST group must each sign to accept their configured role before the protocol becomes active.

Remediation

This issue has been acknowledged by Bulk Trade, and a fix was implemented in commit [c25093ff](#).

3.6. max_balance does not impose a protocol-wide deposit cap

Target	PreDepositConfig		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The predeposit max_balance value is configured for each vault, but it is enforced against the balance recorded in an individual wallet's PreDeposit account. Each such account is derived using the payer address, so separate wallets receive separate allocations for the same vault.

The predeposit instruction derives the state account from the payer and applies the configured limit to that wallet's vault entry:

```
derive_and_check(
    pre_deposit,
    &[PreDeposit::SEED.as_ref(), payer.address().as_ref()],
    program_id,
)?;

let state = unsafe {
    load_mut::<PreDeposit>(pre_deposit.borrow_unchecked_mut())? };

if state.user != *payer.address() {
    return Err(ProgramError::InvalidAccountOwner);
}

upsert_vault_entry(state, &config, vault_idx, vault.address(), amount)?;
```

The limit check adds the requested amount only to the balance obtained from that wallet's selected vault entry:

```
let new_balance = current_balance
    .checked_add(amount)
    .ok_or(ProgramError::ArithmeticOverflow)?;

let max_balance = self.max_balance(idx);
if max_balance != 0 && new_balance > max_balance {
    return Err(ProgramError::Custom(246));
}
```

Each wallet has a separate `current_balance` and can therefore deposit up to the same limit. This behavior is consistent with a per-wallet, per-vault allocation and does not represent a bypass of that allocation model.

The separate regular deposit instruction also remains available without consulting `PreDepositConfig` or the predeposit limit:

```
pub fn process_deposit(  
    program_id: &Address,  
    accounts: &mut [AccountView],  
    data: &[u8],  
) -> ProgramResult {  
    // ...  
  
    let amount = parse_amount(data)?;  
  
    Transfer::new(user_token_account, vault_token_account, payer,  
        amount).invoke()?;  
  
    Ok(())  
}
```

Regular deposits do not update a `PreDeposit` entry or grant a corresponding predeposit withdrawal entitlement. They therefore do not bypass the intended per-wallet allocation, but they do mean that `max_balance` cannot be interpreted as a cap on total vault or protocol inflows.

Impact

Integrators or operators could incorrectly interpret `max_balance` as a protocol-wide exposure limit and rely on it to restrict total inflows, even though it only limits each wallet's predeposit balance for a vault.

Recommendations

Document `max_balance` as a per-wallet, per-vault predeposit allocation and state that regular deposits are outside this limit.

Remediation

This issue has been acknowledged by Bulk Trade.

Bulk Trade stated that the allocation limit is intentionally per wallet rather than a global deposit cap.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Code quality and implementation simplicity

The codebase is generally well written. Its custody model and core business logic are compact: deposits place tokens into mint-specific vaults, while withdrawals rely on the configured FROST authority. The program also consistently applies explicit PDA, owner, token-program, mint, and account-size checks at its instruction boundaries. This relatively simple design makes the program's state transitions and security assumptions easier to reason about and test.

4.2. Narrow mutable borrows and reference scopes

`initialize_vault` takes a mutable account-data borrow of the global Bulk state solely to perform an authorization check. An immutable borrow is sufficient for this read-only operation. More generally, constraining account references and zero-copy borrows to the smallest practical scope makes aliasing assumptions clearer and simplifies review of unsafe account-data access.

4.3. Remove the unused `set_frost_group_key` helper

The `set_frost_group_key` helper is not used by the current program flow. If it is not reserved for an imminent change, removing it would reduce unused code and make the authoritative key-update path easier to identify and maintain.

5. Threat Model

As time permitted, we analyzed each instruction in the program and created a written threat model for the most critical instructions. A threat model documents the high-level functionality of a given instruction, the inputs it receives, and the accounts it operates on as well as the main checks performed on them; it gives an overview of the attack surface of the programs and of the level of control an attacker has over the inputs of critical instructions.

Not all instructions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that an instruction is safe.

5.1. Program: bulk-program

Instruction: initialize

This instruction performs the one-time initialization of the global Bulk configuration. It records the FROST group public key that authorizes withdrawals and the persistent administrator key. Only the account currently authorized to upgrade the deployed program may initialize it.

Input parameters

- `frost_group_key` (bytes 0–31): The initial FROST group public key.
- `admin_key` (bytes 32–63): The persistent administrative public key.

The payload must be exactly 64 bytes. Neither address is required to be nonzero or to prove control during initialization.

Accounts

- **payer**: The current program upgrade authority and rent payer for the global state.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
 - Constraints: Its address must equal the upgrade-authority address encoded in `program_data`. If `state` requires a rent top-up, `payer` must be owned by the System Program, carry no data, and have sufficient lamports to fund the deficit.
- **state**: The global Bulk configuration account.
 - Signer: No.
 - Init: Yes.
 - PDA: Yes, seeds [`Bulk` : `SEED`].

- Mutable: Yes.
- Constraints: Must be the canonical PDA, data-empty, and owned by the System Program before initialization. Lamport prefunding is allowed, but the account is allocated to exactly `Bulk : LEN`, assigned to this program, and funded to the rent-exempt minimum.
- **program_data**: Solana's deployment-metadata account for this program. It records which account currently holds the program's upgrade authority.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds `[program_id]`.
 - Mutable: No.
 - Constraints: Must be the authentic deployment-metadata account for this program, have the expected metadata format, and contain an upgrade-authority address matching payer.
- **system_program**: The System Program used to fund, allocate, and assign state.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Its address must equal the native System Program ID.

Additional checks and behavior

- The instruction requires `program_data` to be derived from `[program_id]` under the Upgradeable Loader, owned by the Upgradeable Loader, and at least 45 bytes long.
- It requires the `program_data` discriminator to equal 3, the upgrade-authority option byte to equal 1, and the stored upgrade-authority address to equal payer.
- It requires `state` to match the PDA derived from `[Bulk : SEED]` and to be data-empty before creation.
- The initialized `Bulk` state stores `Bulk : SEED` as its discriminator, the two addresses supplied in the instruction data, and 64 zeroed reserved bytes.

CPI

- Invokes the System Program to transfer any missing rent, allocate the canonical `state` PDA, and assign it to this program. The program supplies the `state` PDA signer seeds.

Test coverage

- `program/tests/initialize.rs` covers successful initialization, the core discriminator

and authority fields, rejection of reinitialization, invalid payload length, missing signatures, an incorrect System Program, an incorrect state PDA, and both underfunded and overfunded prefunded PDAs.

- It also covers an unauthorized upgrade authority, missing upgrade-authority metadata, and an incorrect deployment-metadata address, owner, or format.

Instruction: `initialize_vault`

This instruction creates the canonical custody `Vault` and SPL Token account for one mint. The FROST group or the administrator selects which classic SPL Token mints the program will custody and funds any rent deficit during initialization.

Input parameters

No input parameters are required.

Accounts

- **signer:** The FROST group or administrator and the rent payer for both new accounts.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
 - Constraints: Its address must equal either `Bulk.frost_group_key` or `Bulk.admin_key`. If either new account requires a rent top-up, signer must be owned by the System Program, carry no data, and have sufficient lamports to fund the required deficit.
- **state:** The global Bulk configuration.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`Bulk::SEED`].
 - Mutable: No.
 - Constraints: Must be the canonical program-owned Bulk account with the exact size and initialized discriminator.
- **vault:** The custody-authority state for mint.
 - Signer: No.
 - Init: Yes.
 - PDA: Yes, seeds [`Vault::SEED`, `mint`].
 - Mutable: Yes.
 - Constraints: Must be the canonical data-empty PDA and owned by the

System Program before initialization. It is allocated to exactly `Vault::LEN`, assigned to this program, and funded to rent exemption.

- **vault_token_account**: The custody token account for mint.
 - Signer: No.
 - Init: Yes.
 - PDA: Yes, seeds `[b"vault_ata", mint]`.
 - Mutable: Yes.
 - Constraints: Must be data-empty and owned by the System Program before creation. It is allocated to the classic SPL Token account size, assigned to the SPL Token Program, and initialized with `vault` as its token authority.
- **mint**: The classic SPL Token mint supported by this vault.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Must be owned by the classic SPL Token Program and have exactly the classic `Mint` data length. For a non-native mint, its serialized `Mint` state must also be initialized. Token-2022 mints are not accepted.
- **system_program**: The System Program used for account funding, allocation, and assignment.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Its address must equal the native System Program ID.
- **token_program**: The program that initializes the custody token account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Its address must equal the classic SPL Token Program ID.

Additional checks and behavior

- The instruction ignores its instruction-data argument.
- It validates state before comparing signer with `Bulk.frost_group_key` and `Bulk.admin_key`.
- It checks that `mint` is owned by the classic SPL Token Program and has exactly `Mint::LEN` bytes of data.

- It requires `vault` and `vault_token_account` to be data-empty and checks their addresses against the PDAs derived using `mint`.
- It initializes `vault_token_account` with `mint` as its mint and `vault` as its token authority.
- The initialized Vault stores `Vault::SEED` as its discriminator, the supplied mint address, the custody token-account address, the vault bump, and 64 zeroed reserved bytes.

CPI

- Invokes the System Program to top up, allocate, and assign both PDAs, using the corresponding PDA signer seeds.
- Invokes the classic SPL Token Program's `InitializeAccount3` instruction to initialize `vault_token_account` for `mint` with `vault` as authority.

Test coverage

- `program/tests/initialize_vault.rs` covers successful creation, prefunded accounts, unauthorized callers, reinitialization of either account, a non-token-owned mint account, incorrect vault and custody-token-account PDAs, and incorrect System or SPL Token Program IDs.
- The suite does not exercise a token-owned mint with an incorrect length or uninitialized internal state. Its instruction builder also marks the off-curve `vault_token_account` PDA as a top-level signer, so it does not faithfully exercise production signer metadata for that account.

Instruction: `deposit`

This permissionless instruction transfers classic SPL Tokens from a signer's token account into the canonical custody account for a mint. It is the main Bulk-network deposit path and does not create or update a program-owned depositor balance or withdrawal claim.

Input parameters

- `amount` (bytes 0–7): A little-endian u64 number of base token units to transfer.

The amount must be nonzero. The handler reads the first eight parameter bytes after the top-level discriminator and ignores any trailing bytes.

Accounts

- **payer:** The depositor and transfer authority for `user_token_account`.
 - Signer: Yes.

- Init: No.
- PDA: No.
- Mutable: No.
- Constraints: Must be the token owner recorded in `user_token_account`.
- **user_token_account**: The source token account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
 - Constraints: Must be a valid classic SPL Token account whose mint equals mint and whose owner equals payer.
- **vault**: The custody-authority state for mint.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`Vault::SEED`, `mint`].
 - Mutable: No.
 - Constraints: Must be owned by this program, have exactly `Vault::LEN`, contain the initialized discriminator, and store both the supplied mint and `vault_token_account` addresses.
- **mint**: The mint of the deposited tokens.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Its address is used to derive `vault`; it must also match the mint stored in both token accounts through the state and token-account checks.
- **vault_token_account**: The canonical custody token account receiving the deposit.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`b"vault_ata"`, `mint`].
 - Mutable: Yes.
 - Constraints: Its address must equal `Vault.token_account`. The SPL Token transfer enforces that it is a valid destination for the same mint.
- **token_program**: The token program used for the transfer.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Its address must equal the classic SPL Token Program ID.

Additional checks and behavior

- The instruction requires payer to sign and requires `user_token_account` to be a classic SPL Token account whose mint equals `mint` and whose token owner equals payer.
- It checks the canonical `vault` address, its program ownership and initialized state, and the mint and custody-token-account addresses stored in `Vault`.
- It reads the first eight instruction-data bytes as amount, rejects zero, and does not require the data to end after those eight bytes.
- It invokes the SPL Token transfer with payer as the transfer authority.
- It does not load a `PreDepositConfig` or `PreDeposit` account and does not write any program-owned state.

CPI

- Invokes the classic SPL Token Program to transfer amount from `user_token_account` to `vault_token_account`, with payer as the transfer authority.

Test coverage

- `program/tests/deposit.rs` covers a successful transfer, zero and undersized amount data, a missing signer, an incorrect token-program address, a source account owned by another address, and custody-token-account substitution.
- The suite does not cover accepted trailing parameter bytes, direct SPL Token transfers into custody, or the absence of an on-chain depositor claim and unique deposit identifier.

Instruction: `withdraw`

This instruction transfers classic SPL Tokens from a canonical custody account to any token account of the same mint. Authorization is provided by a transaction signature from the public key stored in `Bulk.frost_group_key`; threshold participation and approval policy are enforced entirely by the off-chain FROST network.

Input parameters

- `amount` (bytes 0–7): A little-endian u64 number of base token units to transfer.

The amount must be nonzero. The handler reads the first eight parameter bytes after the top-level discriminator and ignores any trailing bytes.

Accounts

- **payer**: The current FROST group public key.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Its address must equal `Bulk.frost_group_key`. The administrator is not accepted directly unless it was first installed as the FROST key.
- **user_token_account**: The withdrawal destination.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
 - Constraints: Must be a valid classic SPL Token account whose mint equals `mint`. Its token owner is intentionally unrestricted, so the FROST-authorized transaction selects the recipient.
- **vault**: The custody authority for the selected mint.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`Vault::SEED`, `mint`].
 - Mutable: No.
 - Constraints: Must be the canonical program-owned initialized `Vault`, and its stored mint and custody-token-account address must match the supplied accounts.
- **vault_token_account**: The custody token account debited by the withdrawal.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`b"vault_ata"`, `mint`].
 - Mutable: Yes.
 - Constraints: Its address must equal `Vault.token_account`; the SPL Token Program validates its state, mint, authority, and balance during the transfer.
- **mint**: The mint being withdrawn.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Its address determines the canonical vault and must equal

`Vault.mint` and the destination token account's mint.

- **state:** The global Bulk configuration containing the current FROST key.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`Bulk : :SEED`].
 - Mutable: No.
 - Constraints: Must be the canonical program-owned Bulk account with the exact size and initialized discriminator.
- **token_program:** The token program used for the transfer.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Its address must equal the classic SPL Token Program ID.

Additional checks and behavior

- The instruction requires payer to sign and requires its address to equal `Bulk.frost_group_key`.
- It checks the canonical state and vault accounts and compares the mint and custody-token-account addresses stored in `Vault` with the supplied accounts.
- It requires `user_token_account` to be a classic SPL Token account whose mint equals `mint`; it does not compare the token owner with payer.
- It reads the first eight instruction-data bytes as amount, rejects zero, and does not require the data to end after those eight bytes.
- It does not compare `user_token_account` and `vault_token_account` for inequality.
- It does not read or write a withdrawal identifier, nonce, receipt, entitlement, amount-limit field, `PreDepositConfig`, or `PreDeposit` account.

CPI

- Invokes the classic SPL Token Program to transfer amount from `vault_token_account` to `user_token_account`. The program signs as `vault` with the mint-bound vault PDA seeds and the bump stored in `Vault`.

Test coverage

- `program/tests/withdraw.rs` covers a successful transfer, a non-FROST signer, zero and undersized amount data, a missing signature, an incorrect token program or

global-state PDA, custody-token-account substitution, and a destination account with the wrong mint.

- The suite does not cover a same-account no-op transfer, business-level duplicate authorization, interaction with outstanding pre-deposit liabilities, accepted trailing bytes, or exhaustion of the shared custody balance.

Instruction: update_frost_group

This instruction replaces the FROST group public key stored in the global configuration. Either the current FROST group or the persistent administrator may perform the rotation.

Input parameters

- `frost_group_key` (bytes 0–31): The replacement FROST group public key.

The payload after the top-level discriminator must be exactly 32 bytes.

Accounts

- **signer:** The authority requesting the rotation.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Its address must equal the current `Bulk.frost_group_key` or the persistent `Bulk.admin_key`.
- **state:** The global Bulk configuration to update.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`Bulk : SEED`].
 - Mutable: Yes.
 - Constraints: Must be the canonical program-owned Bulk account with exactly `Bulk : LEN` and the initialized discriminator.

Additional checks and behavior

- The instruction validates the canonical initialized Bulk account and requires signer to match the current `Bulk.frost_group_key` or `Bulk.admin_key`.
- It requires exactly 32 bytes of instruction data and converts all 32 bytes into the replacement address.

- It does not perform any additional validation on the replacement address and does not require a signature from that address.
- It writes the replacement address to `Bulk.frost_group_key`.
- It does not modify `Bulk.admin_key`, the discriminator, or the reserved bytes.

CPI

- No cross-program invocations are performed.

Test coverage

- `program/tests/update_frost_group.rs` covers successful rotation by the FROST group and by the administrator, an unauthorized signer, short and trailing payload data, and an invalid global state.
- The suite does not cover zero or otherwise uncontrollable replacement keys, concurrent operational key rotation, or a two-step acceptance flow because the implementation has no such validation.

Instruction: `initialize_predeposit_config`

This instruction performs the one-time initialization of the global pre-deposit configuration. The account stores up to 16 approved vault addresses together with their activity flags and per-wallet deposit limits.

Input parameters

No input parameters are required. The handler does not validate or interpret bytes that follow the top-level 255 discriminator and pre-deposit subinstruction discriminator 0.

Accounts

- **signer:** The FROST group or administrator and the rent payer for the new configuration.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
 - Constraints: Its address must equal `Bulk.frost_group_key` or `Bulk.admin_key`. If `pre_deposit_config` requires a rent top-up, signer must be owned by the System Program, carry no data, and have sufficient lamports to fund the deficit.

- **state:** The global Bulk configuration used for authorization.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [Bulk::SEED].
 - Mutable: No.
 - Constraints: Must be the canonical program-owned initialized Bulk account.
- **pre_deposit_config:** The global pre-deposit configuration.
 - Signer: No.
 - Init: Yes.
 - PDA: Yes, seeds [PreDepositConfig::SEED].
 - Mutable: Yes.
 - Constraints: Must be the canonical data-empty PDA and owned by the System Program before initialization. It is funded to rent exemption, allocated to exactly PreDepositConfig::LEN, and assigned to this program.
- **system_program:** The System Program used for account creation.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Its address must equal the native System Program ID.

Additional checks and behavior

- The instruction ignores its instruction-data argument.
- It validates the canonical program-owned Bulk state, then requires signer to equal Bulk.frost_group_key or Bulk.admin_key.
- It derives pre_deposit_config from [PreDepositConfig::SEED] and rejects the account if its data is nonempty.
- It creates the PDA with PreDepositConfig::LEN, writes PreDepositConfig::SEED to discriminator, and sets vault_count to zero.

CPI

- Invokes the System Program to transfer any missing rent, allocate the pre_deposit_config PDA, and assign it to this program, using the PDA signer seeds.

Test coverage

- program/tests/predeposit/initialize_config.rs covers successful initialization,

reinitialization rejection, unauthorized callers, an incorrect configuration PDA, and an incorrect System Program ID.

- No test supplies trailing payload bytes, although the implementation accepts them.

Instruction: `add_predeposit_vault`

This privileged instruction registers an existing custody vault in the global pre-deposit configuration. It sets the vault's initial activity flag and, optionally, its minimum per-transaction deposit and maximum per-wallet recorded balance.

Input parameters

- `flags` (byte 0): A one-byte bit field. Only the `VAULT_FLAG_ACTIVE` bit (1 << 0) is defined, so the accepted values are 0 and 1.
- `min_per_tx` (bytes 1–8, optional): A little-endian u64; zero disables the minimum.
- `max_balance` (bytes 9–16, optional): A little-endian u64; zero disables the maximum.

A one-byte payload sets both limits to zero. A 17-byte payload sets both supplied limits; all other lengths are rejected.

Accounts

- **signer:** The authority adding the vault.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Its address must equal `Bulk.frost_group_key` or `Bulk.admin_key`.
- **state:** The global Bulk configuration used for authorization.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`Bulk::SEED`].
 - Mutable: No.
 - Constraints: Must be the canonical program-owned initialized Bulk account.
- **pre_deposit_config:** The configuration receiving the new vault entry.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`PreDepositConfig::SEED`].
 - Mutable: Yes.

- Constraints: Must be owned by this program, have exactly `PreDepositConfig::LEN`, and contain the initialized discriminator.
- **vault**: The existing custody vault to register.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds `[Vault::SEED, mint]`.
 - Mutable: No.
 - Constraints: Must be the canonical program-owned initialized `Vault` for `mint` and must not already appear in the configuration.
- **mint**: The classic SPL Token mint associated with `vault`.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Must be owned by the classic SPL Token Program and have exactly the classic `Mint` data length; its address determines the expected vault PDA.

Additional checks and behavior

- The instruction rejects the call when `vault_count` is equal to or greater than the 16-element vault array length.
- It rejects a vault address already present among the configured entries and rejects flag bits other than `VAULT_FLAG_ACTIVE`.
- If both limits are nonzero, it rejects `min_per_tx` greater than `max_balance`. A one-byte payload stores zero for both limits.
- On success, it writes the vault address, flags, and limits at `vault_count`, then increments `vault_count` with checked arithmetic.

CPI

- No cross-program invocations are performed.

Test coverage

- `program/tests/predeposit/add_vault.rs` covers successful registration, duplicate registration, the 16-vault capacity bound, invalid flag bits, inconsistent minimum and maximum limits, a vault/mint mismatch, and an unauthorized signer.
- The suite does not exercise the one-byte payload that leaves both limits unset or the long-term exhaustion of nonremovable configuration slots.

Instruction: `initialize_predeposit_account`

This permissionless instruction creates the calling user's pre-deposit ledger account. The ledger can hold one recorded claim per configured vault, up to 16 entries.

Input parameters

No input parameters are required. The handler does not validate or interpret bytes that follow the top-level 255 discriminator and pre-deposit subinstruction discriminator 2.

Accounts

- **signer:** The user whose ledger is created and the account-creation rent payer.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
 - Constraints: Its address is incorporated into the `pre_deposit` PDA and stored as `PreDeposit.user`. If `pre_deposit` requires a rent top-up, `signer` must be owned by the System Program, carry no data, and have sufficient lamports to fund the deficit.
- **pre_deposit:** The user's pre-deposit ledger.
 - Signer: No.
 - Init: Yes.
 - PDA: Yes, seeds [`PreDeposit::SEED`, `signer`].
 - Mutable: Yes.
 - Constraints: Must be the canonical data-empty PDA and owned by the System Program before initialization. It is funded to rent exemption, allocated to exactly `PreDeposit::LEN`, and assigned to this program.
- **system_program:** The System Program used for account creation.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Its address must equal the native System Program ID.

Additional checks and behavior

- The instruction ignores its instruction-data argument.

- It checks that `signer` signed the transaction and that `system_program` has the native System Program address.
- It derives `pre_deposit` from `[PreDeposit::SEED, signer]` and rejects the account if its data is nonempty.
- It creates the PDA with `PreDeposit::LEN`, writes `PreDeposit::SEED` to discriminator, stores `signer` in `user`, and sets `entry_count` to zero.
- It does not load or validate a `PreDepositConfig` account or a `Vault` account.

CPI

- Invokes the System Program to transfer any missing rent, allocate the user-specific `pre_deposit` PDA, and assign it to this program, using the PDA signer seeds.

Test coverage

- `program/tests/predeposit/initialize_account.rs` covers successful initialization, lamport-prefunded PDAs, reinitialization rejection, an incorrect PDA, an incorrect System Program ID, and a missing signature.
- No test supplies trailing payload bytes, although the implementation accepts them.

Instruction: `predeposit_deposit`

This instruction transfers classic SPL Tokens from a user into a configured custody vault and atomically increases that user's on-chain `PreDeposit` claim for the vault. Unlike the main `deposit` instruction, it records the user's entitlement on-chain.

Input parameters

- `amount` (bytes 0–7): A little-endian u64 number of base token units to transfer and credit.

The amount must be nonzero. The handler reads the first eight parameter bytes after the two routing discriminators and ignores any trailing bytes.

Accounts

- **`payer`:** The depositor, source-token authority, and owner of the ledger being credited.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: No.

- Constraints: Must equal both the owner of `user_token_account` and `PreDeposit.user`; its address is also a seed of `pre_deposit`.
- **user_token_account**: The source token account.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
 - Constraints: Must be a valid classic SPL Token account whose mint equals mint and whose owner equals payer.
- **vault**: The custody-authority state receiving the deposit.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`Vault::SEED`, `mint`].
 - Mutable: No.
 - Constraints: Must be the canonical program-owned initialized `Vault`, be registered in `pre_deposit_config`, have `VAULT_FLAG_ACTIVE`, and store the supplied mint and custody-token-account addresses.
- **vault_token_account**: The shared custody token account receiving the assets.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`"vault_ata"`, `mint`].
 - Mutable: Yes.
 - Constraints: Its address must equal `Vault.token_account`; the SPL Token transfer validates it as a destination for the same mint.
- **mint**: The mint being deposited.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Its address determines the canonical vault and must match the source and custody token accounts.
- **pre_deposit_config**: The global vault allowlist, activity flags, and deposit limits.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`PreDepositConfig::SEED`].
 - Mutable: No.
 - Constraints: Must be the canonical program-owned initialized `PreDepositConfig` and contain `vault` among its first `vault_count` entries.
- **pre_deposit**: The user's ledger to credit.

- Signer: No.
- Init: No.
- PDA: Yes, seeds [`PreDeposit::SEED`, `payer`].
- Mutable: Yes.
- Constraints: Must be owned by this program, have exactly `PreDeposit::LEN`, contain the initialized discriminator, and store `payer` as its user.
- **token_program**: The token program used for the transfer.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Its address must equal the classic SPL Token Program ID.

Additional checks and behavior

- The instruction rejects a vault that is absent from `pre_deposit_config` or does not have `VAULT_FLAG_ACTIVE`.
- It rejects `entry_count` values greater than the 16-element entry array length.
- For an existing vault entry, it adds amount with checked arithmetic and writes the resulting balance. For a new vault entry, it rejects a full entry array, writes the entry at `entry_count`, and increments `entry_count` with checked arithmetic.
- A nonzero `min_per_tx` rejects a deposit below that value. A nonzero `max_balance` rejects a resulting vault-entry balance above that value.
- The instruction updates the `PreDeposit` entry before invoking the SPL Token transfer.

CPI

- Invokes the classic SPL Token Program to transfer amount from `user_token_account` to `vault_token_account`, with `payer` as authority.

Test coverage

- `program/tests/predeposit/deposit.rs` covers new and existing entries; zero and undersized data; inactive and unregistered vaults; minimum, maximum, arithmetic-overflow, and entry-capacity bounds; corrupt entry counts; an incorrect ledger owner or PDA; invalid source ownership; custody-account substitution; an incorrect token program; and attempted self-transfer inflation.
- The suite does not cover a corrupted `PreDeposit.user` field at the canonical PDA, a source token account with only the wrong mint, accepted trailing parameter bytes, a

multi-wallet bypass of `max_balance`, or FROST withdrawal of assets backing outstanding claims.

Instruction: `predeposit_withdraw`

This instruction redeems part or all of a user's recorded pre-deposit claim. It atomically decreases the claim and transfers the same amount of classic SPL Tokens from the shared custody account back to a token account owned by that user.

Input parameters

- `amount` (bytes 0–7): A little-endian u64 number of base token units to debit and transfer.

The amount must be nonzero. The handler reads the first eight parameter bytes after the two routing discriminators and ignores any trailing bytes.

Accounts

- **`payer`**: The withdrawing user and owner of both the ledger and destination token account.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Must equal `PreDeposit.user` and the owner of `user_token_account`; its address is also a seed of `pre_deposit`.
- **`user_token_account`**: The withdrawal destination.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
 - Constraints: Must be a valid classic SPL Token account whose mint equals `mint` and whose owner equals `payer`.
- **`vault`**: The custody authority for the redeemed claim.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`Vault::SEED`, `mint`].
 - Mutable: No.
 - Constraints: Must be the canonical program-owned initialized `Vault`, be registered and active in `pre_deposit_config`, and store the supplied mint

and custody-token-account addresses.

- **vault_token_account**: The shared custody token account funding the redemption.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [b"vault_ata", mint].
 - Mutable: Yes.
 - Constraints: Its address must equal Vault.token_account; the SPL Token Program validates its state, mint, authority, and available balance during the transfer.
- **mint**: The mint being withdrawn.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Its address determines the canonical vault and must match both token accounts and Vault.mint.
- **pre_deposit_config**: The global vault allowlist and activity configuration.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [PreDepositConfig::SEED].
 - Mutable: No.
 - Constraints: Must be the canonical program-owned initialized PreDepositConfig, contain vault, and mark it with VAULT_FLAG_ACTIVE.
- **pre_deposit**: The user's ledger to debit.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [PreDeposit::SEED, payer].
 - Mutable: Yes.
 - Constraints: Must be owned by this program, have exactly PreDeposit::LEN, contain the initialized discriminator, store payer as its user, and contain a vault entry with at least amount credited.
- **token_program**: The token program used for the transfer.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Its address must equal the classic SPL Token Program ID.

Additional checks and behavior

- The instruction rejects a vault that is absent from `pre_deposit_config` or does not have `VAULT_FLAG_ACTIVE`.
- It rejects `entry_count` values greater than the 16-element entry array length, a missing vault entry, and an amount greater than the recorded entry amount.
- If an entry remains nonzero after subtraction, it writes the remaining amount. If the remaining amount is zero, it moves the last active entry into the removed entry's slot, clears the last slot, and decrements `entry_count`.
- The instruction updates the `PreDeposit` entry before invoking the SPL Token transfer.

CPI

- Invokes the classic SPL Token Program to transfer amount from `vault_token_account` to `user_token_account`. The program signs as `vault` with the mint-bound vault PDA seeds and stored bump.

Test coverage

- `program/tests/predeposit/withdraw.rs` covers partial and full redemption, removal of the last entry, inactive vaults, zero and undersized data, absent and insufficient claims, corrupt entry counts, an incorrect ledger owner or PDA, custody-account substitution, an invalid destination owner, and an incorrect token program.
- The suite does not cover a corrupted `PreDeposit.user` field at the canonical PDA, removal of a nonfinal entry, a destination token account with only the wrong mint, custody insolvency caused by the main withdrawal path, accepted trailing parameter bytes, or a user exit after privileged deactivation.

Instruction: `set_predeposit_vault_flags`

This privileged instruction replaces the activity flags for one registered pre-deposit vault. The only defined flag, `VAULT_FLAG_ACTIVE`, simultaneously enables or disables new pre-deposits and user redemptions.

Input parameters

- `flags` (byte 0): The replacement one-byte bit field. The only accepted values are 0 and `VAULT_FLAG_ACTIVE` (`1 << 0`).

The payload after the two routing discriminators must be exactly one byte.

Accounts

- **signer**: The authority updating the flag.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Its address must equal `Bulk.frost_group_key` or `Bulk.admin_key`.
- **state**: The global Bulk configuration used for authorization.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`Bulk::SEED`].
 - Mutable: No.
 - Constraints: Must be the canonical program-owned initialized Bulk account.
- **pre_deposit_config**: The global configuration containing the flag.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`PreDepositConfig::SEED`].
 - Mutable: Yes.
 - Constraints: Must be the canonical program-owned initialized `PreDepositConfig` and contain `vault` among its configured entries.
- **vault**: The registered vault whose flag is updated.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`Vault::SEED`, `mint`].
 - Mutable: No.
 - Constraints: Must be the canonical program-owned initialized `Vault` for `mint` and already appear in `pre_deposit_config`.
- **mint**: The classic SPL Token mint associated with vault.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Must be owned by the classic SPL Token Program and have the exact classic `Mint` data length; its address determines the expected vault PDA.

Additional checks and behavior

- The instruction requires exactly one byte of instruction data and rejects flag bits other than `VAULT_FLAG_ACTIVE`.
- It validates the supplied vault against mint and requires the vault address to be present in `pre_deposit_config`.
- It overwrites the registered vault's flag byte with the supplied value.
- It does not modify the registered vault address or its deposit limits.

CPI

- No cross-program invocations are performed.

Test coverage

- `program/tests/predeposit/set_vault_flags.rs` covers a successful update, invalid flag bits, an unregistered vault, an unauthorized signer, incorrect configuration ownership, and a vault/mint mismatch.
- Cross-instruction blocking behavior is covered in `program/tests/predeposit/disable_all_vault_activity.rs`, but the suite does not cover an outstanding claimant's full deactivate/reactivate lifecycle or migration replay after reactivation.

Instruction: `set_predeposit_vault_deposit_limits`

This privileged instruction replaces the minimum per-transaction deposit and maximum recorded balance for one registered pre-deposit vault.

Input parameters

- `min_per_tx` (bytes 0–7): A little-endian u64; zero disables the minimum deposit amount.
- `max_balance` (bytes 8–15): A little-endian u64; zero disables the maximum recorded balance.

The payload after the two routing discriminators must be exactly 16 bytes. If both values are nonzero, `min_per_tx` must not exceed `max_balance`.

Accounts

- **signer:** The authority updating the limits.
 - Signer: Yes.

- Init: No.
- PDA: No.
- Mutable: No.
- Constraints: Its address must equal `Bulk.frost_group_key` or `Bulk.admin_key`.
- **state**: The global Bulk configuration used for authorization.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`Bulk::SEED`].
 - Mutable: No.
 - Constraints: Must be the canonical program-owned initialized Bulk account.
- **pre_deposit_config**: The global configuration containing the vault limits.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`PreDepositConfig::SEED`].
 - Mutable: Yes.
 - Constraints: Must be the canonical program-owned initialized `PreDepositConfig` and contain `vault` among its configured entries.
- **vault**: The registered vault whose limits are updated.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`Vault::SEED`, `mint`].
 - Mutable: No.
 - Constraints: Must be the canonical program-owned initialized `Vault` for `mint` and already appear in `pre_deposit_config`.
- **mint**: The classic SPL Token mint associated with `vault`.
 - Signer: No.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Must be owned by the classic SPL Token Program and have the exact classic `Mint` data length; its address determines the expected vault PDA.

Additional checks and behavior

- The instruction requires exactly 16 bytes of instruction data and reads `min_per_tx` and `max_balance` as little-endian u64 values.
- It validates the supplied `vault` against `mint` and requires the vault address to be

present in `pre_deposit_config`.

- If both values are nonzero, it rejects `min_per_tx` greater than `max_balance`.
- It overwrites the registered vault's stored `min_per_tx` and `max_balance` values.
- It does not read or modify any `PreDeposit` account.

CPI

- No cross-program invocations are performed.

Test coverage

- `program/tests/predeposit/set_vault_deposit_limits.rs` covers successful updates, inconsistent minimum and maximum values, an unregistered vault, an unauthorized signer, incorrect configuration ownership, and enforcement by a later deposit.
- The suite does not cover setting limits below an existing claim, zero-as-unset behavior, multi-wallet limit bypass, or interaction with the unrestricted main deposit path.

Instruction: `predeposit_close_account`

This instruction lets a user close their empty pre-deposit ledger and reclaim all of its lamports. It cannot close a ledger while `entry_count` is nonzero.

Input parameters

No input parameters are required. The handler does not validate or interpret bytes that follow the top-level 255 discriminator and pre-deposit subinstruction discriminator 7.

Accounts

- **signer:** The ledger owner and recipient of the reclaimed lamports.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: Yes.
 - Constraints: Its address must equal `PreDeposit.user` and is used to derive the expected `pre_deposit` PDA.
- **pre_deposit:** The empty user ledger to close.
 - Signer: No.

- Init: No.
- PDA: Yes, seeds `[PreDeposit::SEED, signer]`.
- Mutable: Yes.
- Constraints: Must be owned by this program, have exactly `PreDeposit::LEN`, contain the initialized discriminator, store the signer as its user, and have `entry_count == 0`.

Additional checks and behavior

- The instruction ignores its instruction-data argument.
- It checks that `pre_deposit` is the program-owned PDA derived from `[PreDeposit::SEED, signer]` and that its stored user equals `signer`.
- It requires `entry_count` to equal zero and does not scan `vault_entries` when evaluating this condition.
- It adds the account's lamports to `signer` with checked arithmetic, sets the account's lamports to zero, and closes its data.

CPI

- No cross-program invocations are performed; lamports are reassigned directly before the program-owned account is closed.

Test coverage

- `program/tests/predeposit/close_account.rs` covers successful closure and rent recovery, rejection of a nonempty ledger, an incorrect signer or PDA, incorrect account ownership, and a missing signature.
- No test supplies trailing payload bytes or exercises closure after a vault has been deactivated with an outstanding claim.

Instruction: `disable_all_predeposit_vault_activity`

This privileged migration-oriented instruction clears `VAULT_FLAG_ACTIVE` for every configured pre-deposit vault. It halts both new pre-deposits and user redemptions but does not stop the main deposit or FROST withdrawal paths.

Input parameters

No input parameters are required. The payload after the top-level 255 discriminator and pre-deposit subinstruction discriminator 8 must be empty.

Accounts

- **signer**: The authority disabling all configured vaults.
 - Signer: Yes.
 - Init: No.
 - PDA: No.
 - Mutable: No.
 - Constraints: Its address must equal `Bulk.frost_group_key` or `Bulk.admin_key`.
- **state**: The global Bulk configuration used for authorization.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`Bulk: :SEED`].
 - Mutable: No.
 - Constraints: Must be the canonical program-owned initialized Bulk account.
- **pre_deposit_config**: The global configuration whose flags are cleared.
 - Signer: No.
 - Init: No.
 - PDA: Yes, seeds [`PreDepositConfig: :SEED`].
 - Mutable: Yes.
 - Constraints: Must be the canonical program-owned initialized `PreDepositConfig`.

Additional checks and behavior

- The instruction requires empty instruction data.
- It validates the canonical program-owned Bulk state and requires `signer` to equal `Bulk.frost_group_key` or `Bulk.admin_key`.
- It iterates over the first `min(vault_count, 16)` flag entries and clears only `VAULT_FLAG_ACTIVE` from each entry.
- It does not modify `vault_count`, vault addresses, deposit limits, or any `PreDeposit` account.

CPI

- No cross-program invocations are performed.

Test coverage

- `program/tests/predeposit/disable_all_vault_activity.rs` covers clearing multiple vault flags, blocking later pre-deposits and withdrawals, unauthorized callers, incorrect configuration ownership, and rejection of nonempty payload data.
- The suite does not cover the end-to-end migration lifecycle, outstanding-ledger closure, continued main-path vault activity during the freeze, or privileged reactivation after off-chain crediting.

6. Assessment Results

During our assessment on the scoped Bulk Program programs, we discovered six findings. No critical issues were found. One finding was of high impact, one was of medium impact, one was of low impact, and the remaining findings were informational in nature.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.